

# カラコンクイズアカデミア | v2.1.1統合構造化仕様書

|                        |                                                                                                                                                                                                 |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 🕒 作成日時                 | @2025年10月6日 17:39                                                                                                                                                                               |
| ➤ DB_yy_ジャンル_プロパティ     | 📑 案件資料, 📁 保存版, </> エンジニアリング                                                                                                                                                                     |
| ➤ DB_取引先データベース (1)     | 🏢 株式会社ホテラバ                                                                                                                                                                                      |
| ➤ DB_プロジェクト管理_PJ管理ツール🔗 | 📦 HL001_オンボーディング用カラコンクイズアプリ開発                                                                                                                                                                   |
| ≡ 中カテ                  | アプリ開発 ツール プロンプト                                                                                                                                                                                 |
| ≡ メモステータス              |                                                                                                                                                                                                 |
| ☑️ メモ確認済み              | <input type="checkbox"/>                                                                                                                                                                        |
| ≡ AI 要約                | カラコンクイズアカデミア v2.1.1の仕様書には、使用ファイルやリソースの一覧、システム全体構造、運用フロー、データ構造定義、機能仕様、API仕様、UI/UXガイドライン、実装詳細が含まれています。特に、クイズ機能やスコアリングの計算方法、誤答抽出の優先順位、エラーハンドリングの対策が詳述されています。また、全リソースの編集権限が統一されており、スムーズな開発と運用が可能です。 |

## カラコンクイズアカデミア v2.1.1 最終仕様書

C:\Users\seran\development\HL001-quiz-karacon-academia\HL001-quiz-karacon-academia-new\docs

最終更新日: 2025年10月6日

プロジェクトコード: HL001

### 📄 目次

1. 使用ファイル・リソース一覧
2. システム全体構造
3. 運用フロー
4. データ構造定義
5. 機能仕様
6. API仕様
7. UI/UXガイドライン
8. 実装詳細 (コード例)

### 1. 使用ファイル・リソース一覧

## 1.1 Googleスプレッドシート

| 名称         | Sheet ID                                     | 主要タブ         | URL                 | 権限  |
|------------|----------------------------------------------|--------------|---------------------|-----|
| 109販促データCP | 1Uf2e0eXwcsQGJFtTtEeAWuYh74lh4fFE4NdjmyKHrj0 | master       | <a href="#">リンク</a> | 編集可 |
| USERS      | 1X0TyeL_1zER6xlceUDSbJX-GFbqvi2orAiSWHRXIC7M | users        | <a href="#">リンク</a> | 編集可 |
| HISTORY    | 1ShWXLvY9RimRYfsAkwoRyM2Bfwj4a3zVmr5bQc33-o0 | quiz_history | <a href="#">リンク</a> | 編集可 |
| RANKINGS   | 1l2REcy2v5OpyzoY3k61kCzJ3SYKOBBCMxTLCeHWutT8 | rankings     | <a href="#">リンク</a> | 編集可 |

## 1.2 GitHubリポジトリ

| 項目        | 内容                                                         |
|-----------|------------------------------------------------------------|
| リポジトリ名    | HL001-quiz-karacon-academia-new                            |
| URL       | https://github.com/y4m4usr/HL001-quiz-karacon-academia-new |
| ユーザー名     | y4m4usr                                                    |
| アクセストークン名 | firefly                                                    |
| トークンコード   | ghp_LPNQCG3VdD2NIBTin91nlCE0P6uKsM1RxQh*                   |
| 権限        | 編集可                                                        |

画像格納パス:

- レンズ画像: [imagesnew1/lens/lens1/](#)
- サムネ画像: [imagesnew1/samune/samune1/](#)

## 1.3 GAS Webアプリ

| 項目      | 内容                                                                                                                    |
|---------|-----------------------------------------------------------------------------------------------------------------------|
| 実行ユーザー  | 自分（開発者アカウント）                                                                                                          |
| アクセス範囲  | 全員（認証不要）                                                                                                              |
| エンドポイント | <a href="https://script.google.com/macros/s/{SCRIPT_ID}/exec">https://script.google.com/macros/s/{SCRIPT_ID}/exec</a> |

# 2. システム全体構造

## 2.1 システム構成図

```
graph TD
    A[Googleスプレッドシート<br/>109販促データCP] --> B[GAS Webアプリ]
    C[GitHub リポジトリ<br/>画像ストレージ] --> B
    B --> D[フロントエンド<br/>HTML/CSS/JavaScript]
    B --> E[スプレッドシート<br/>USERS/HISTORY/RANKINGS]
```

style A fill:#b7e4b7,stroke:#2b3a2b,color:#1a1a1a  
style C fill:#ffb3b3,stroke:#3a2b2b,color:#1a1a1a  
style B fill:#b3d4ff,stroke:#2b3340,color:#1a1a1a  
style D fill:#ffd199,stroke:#3a2b1f,color:#1a1a1a  
style E fill:#d1b3ff,stroke:#2f2640,color:#1a1a1a

データフロー:

1. 問題取得: GitHub画像 → GAS → フロント
2. 回答送信: フロント → GAS → HISTORY

### 3. スコア計算: GAS (採点) → RANKINGS更新

## 2.2 データフロー詳細

### 【問題生成フロー】

```
sequenceDiagram
    participant F as フロント
    participant G as GAS
    participant S as 109販促CP
    participant GH as GitHub

    F->>G: GET /items
    G->>S: 商品データ読み込み<br/>(E,I,J,K,AJ等)
    S->>G: 行データ返却
    G->>G: 10問生成<br/>E||J重複除外<br/>誤答3件抽出
    G->>G: GitHub URL生成
    G->>F: 問題JSON<br/>(画像URL含む)
    F->>GH: 画像取得
    GH->>F: 画像データ
```

### 【成績保存フロー】

```
sequenceDiagram
    participant F as フロント
    participant G as GAS
    participant H as HISTORY
    participant R as RANKINGS

    F->>G: POST /submit<br/>(スコア等)
    G->>H: quiz_history に行追記
    H->>G: 保存完了
    G->>R: rankings 更新
    R->>G: 更新完了
    G->>F: success: true
```

## 2.3 権限設定

### 全リソース編集権限統一済み

| リソース                   | 権限      | 備考            |
|------------------------|---------|---------------|
| 109販促データCP             | 編集可     | 商品マスタの直接更新が可能 |
| USERS/HISTORY/RANKINGS | 編集可     | GASから直接書き込み   |
| GitHub                 | 編集可     | 画像の手動追加が可能    |
| GAS Webアプリ             | 全員アクセス可 | 認証不要で実行       |

## 3. 運用フロー

### 3.1 画面遷移フロー（修正版）

```
graph TD
    A[開始] → B[ログイン画面]
```

B → C[番号・ユーザー名入力]  
C → D{入力情報<br/>存在する?}  
D → |存在しない| E[エラーメッセージ表示<br/>「入力情報が存在しません」]  
E → C  
D → |存在する| F[ホーム画面]  
F → G[本番モード<br/>10問クイズ]  
F → H[練習モード<br/>10問クイズ]  
G → I[採点結果]  
H → I  
F → J[ランキング]  
F → K[マイ成績]

## 修正内容の詳細

### 1. ログイン画面の変更点

入力項目:

- 番号入力欄: スタッフ番号を入力
- ユーザー名入力欄: ユーザー名を入力

動作仕様:

- 両方の情報を入力後、「ログイン」ボタン押下
- 入力情報がデータベース（USERSシート）に存在するか照合

成功時:

-  入力情報が一致 → **ホーム画面へ遷移**

失敗時:

-  入力情報が存在しない → **エラーメッセージ表示**
  - メッセージ: 「入力情報が存在しません」
  - 入力欄はクリアせず、すぐに修正・再入力可能な状態を維持
  - 同じログイン画面内でエラー表示（別画面への遷移なし）

### 2. ホーム画面への遷移

遷移条件:

- 番号とユーザー名の組み合わせがUSERSシートに存在する場合のみ

遷移後の動作:

- ホーム画面に表示されるユーザー情報（名前・所属店舗・レベル・ポイント等）を取得
- セッション情報として保持

## 削除された機能

以下の画面・機能は削除されました：

-  「アカウントあり or なし」の分岐判定
-  新規アカウント登録画面
-  新規アカウント登録成功画面

→ ユーザー登録は事前に管理者が行う運用に変更

## 3.2 クイズ回答フロー

```
graph LR
  A[問題表示<br/>レンズ画像×2] --> B{ヒント<br/>使用?}
  B -->|Yes| C[ヒント表示<br/>-3%減点]
  B -->|No| D[回答選択]
  C --> D
  D --> E[送信]
  E --> F{正解?}
  F -->|Yes| G[正解フィードバック<br/>サムネ画像表示<br/>2秒]
  F -->|No| H[不正解フィードバック<br/>正解表示<br/>2秒]
  G --> I{最終問題?}
  H --> I
  I -->|No| A
  I -->|Yes| J[結果画面<br/>総合スコア]
```

## 4. データ構造定義

### 4.1 スプレッドシート列定義

#### 109販促データCP（master タブ）

| 列  | 列名       | 用途            | 必須 | 備考                    |
|----|----------|---------------|----|-----------------------|
| E  | 元品番      | 複合キーE         | ✓  | 商品の一意識別子              |
| I  | ブランド（カナ） | 複合キーI / 表示名   | ✓  | 例: ダスティティー            |
| J  | カラー（カナ）  | 複合キーJ / 表示名   | ✓  | 例: クレイジーブラウン          |
| K  | 装用期間     | 複合キーK / ファイル名 | ✓  | 例: 1day, 1month       |
| P  | DIA      | ヒント1          | 任意 | 直径                    |
| Q  | G.DIA    | ヒント1          | 任意 | 着色直径                  |
| R  | BC       | ヒント1          | 任意 | ベースカーブ                |
| AK | コメント     | ヒント2          | 任意 | 商品説明文                 |
| AJ | カラーカテゴリ  | 誤答選定          | ✓  | カンマ区切り（例: ブラウン, ベージュ） |

#### データ前提:

- **2行目:** ヘッダ（項目行）
- **3行目以降:** データ本体
- **空白セル:** に置換済み（を含む行は除外対象）

### 4.2 画像ファイル命名規則

#### 形式:

E\_I\_J\_K\_{lens|samune}.jpg

#### 例:

ABC123\_ダスティティー\_クレイジーブラウン\_1day\_lens.jpg  
ABC123\_ダスティティー\_クレイジーブラウン\_1day\_samune.jpg

#### 命名規則の構成要素:

- **E**: 元品番
- **I**: ブランド名 (カナ)
- **J**: カラー名 (カナ)
- **K**: 装用期間
- **{lens|samune}**: 画像タイプ
  - **lens**: 問題表示用 (左右2枚表示)
  - **samune**: 正解フィードバック用

### 4.3 複合キー (重複除外ルール)

完全一意キー: **E|I|J|K**

重複除外キー: **E|I|J** (装用期間Kを除外)

理由:

- **E|I|J** が一致し **K** のみ異なる場合、デザインは同一
- 1回の出題で同一デザインが複数出ないよう **E|I|J** で判定

重複除外例:

| E      | I        | J         | K      | 採用判定     |
|--------|----------|-----------|--------|----------|
| ABC123 | ダスティティ   | クレイジーブラウン | 1day   | ✓ 採用     |
| ABC123 | ダスティティ   | クレイジーブラウン | 1month | ✗ 除外 (E) |
| ABC456 | アイクローゼット | ホロブラウン    | 1day   | ✓ 採用     |

### 4.4 ステップ4: 誤答3件の抽出 (修正版)

抽出条件 (優先順位順)

【第1優先】カテゴリ一致候補

- 条件:
  1. **E|I|J** が正答と異なる
  2. AJ列のカテゴリが正答と1つ以上一致
  3. 3件見つかるまでランダム抽出

【第2優先】フォールバック (カテゴリ無視)

- カテゴリ一致候補が3件未満の場合:
  1. カテゴリ条件を無視
  2. **E|I|J** が正答および既選択の誤答と異なる
  3. ランダムに不足分を抽出

【第3優先】完全ランダム抽出 (新規追加) ★

- 第2優先でも3件未満の場合:
  1. **E|I|J** の重複チェックのみ実施 (正答との重複除外)
  2. 全商品からランダムに不足分を抽出
  3. データが極端に少ない場合、装用期間K違いの同一商品 (**E|I|J**一致) も許容

## 5. 機能仕様

### 5.1 クイズ機能

| 項目    | 内容                                                                                                            |
|-------|---------------------------------------------------------------------------------------------------------------|
| 認証    | スタッフ番号+名前（暫定）                                                                                                 |
| モード   | 本番（1日1回）／練習（無制限）                                                                                              |
| 問題数   | 10問固定                                                                                                         |
| 制限時間  | 20秒/問                                                                                                         |
| 出題対象  | データ完備商品のみ（E,I,J,K,AJ が  でない） |
| ヒント   | ①DIA/G.DIA/BC、②コメント                                                                                           |
| ヒント減点 | 各 -3%                                                                                                         |
| 画像表示  | レンズ画像を左右2枚                                                                                                    |

## 5.2 スコアリング

計算式:

スコア = 100% - (不正解数 × 10%) - (ヒント使用数 × 3%)  
 下限: 0%  
 上限: 100%

タイムアウト: 未回答扱い（-10%相当）

正解フィードバック:

- 表示時間: 2秒
- 画像: サムネ画像（samune）を表示
- 自動遷移: 次の問題へ

## 5.3 ランキング機能

期間切り替え: 日/週/月

表示順:

1. スコア合計 desc
2. 正答率 desc
3. 試行回数 desc
4. 最終回答日時 desc

掲載条件: attempts >= 3（3回以上回答したユーザーのみ）

## 5.4 マイ成績

表示内容:

- 直近10回の成績（スコア・正答率・時間）
- 通算サマリ（総回数・平均スコア・平均時間）
- Canvas描画のグラフ（ポイント推移・正答率推移）

# 6. API仕様

## 6.1 エンドポイント一覧

GAS Webアプリ URL: [https://script.google.com/macros/s/{SCRIPT\\_ID}/exec](https://script.google.com/macros/s/{SCRIPT_ID}/exec)

| メソッド | エンドポイント                                    | 説明          |
|------|--------------------------------------------|-------------|
| GET  | ?action=items                              | 10問の出題候補取得  |
| POST | action=submit                              | 回答結果送信・履歴保存 |
| GET  | ?action=ranking&scope=daily weekly monthly | ランキング取得     |
| GET  | ?action=mystats&user={user_id}             | マイ成績取得      |

## 6.2 API詳細

【GET】問題取得: ?action=items

レスポンス例:

```
{
  "questions": [
    {
      "questionNumber": 1,
      "lensImageUrl": "https://raw.githubusercontent.com/y4m4usr/HL001-quiz-karacon-academia-new/main/imagesnew1/lens/lens1/ABC123_ダスティティー_クレイジーブラウン_1day_lens.jpg",
      "thumbnailImageUrl": "https://raw.githubusercontent.com/y4m4usr/HL001-quiz-karacon-academia-new/main/imagesnew1/samune/samune1/ABC123_ダスティティー_クレイジーブラウン_1day_samune.jpg",
      "correctAnswer": {
        "E": "ABC123",
        "I": "ダスティティー",
        "J": "クレイジーブラウン",
        "K": "1day"
      },
      "options": [
        {
          "id": 1,
          "brandName": "ダスティティー",
          "colorName": "クレイジーブラウン",
          "isCorrect": true
        },
        {
          "id": 2,
          "brandName": "アイクローゼット",
          "colorName": "ホロブラウン",
          "isCorrect": false
        },
        {
          "id": 3,
          "brandName": "ハニーキス",
          "colorName": "ちゅるんブラウン",
          "isCorrect": false
        },
        {
          "id": 4,
          "brandName": "ミークレール",
          "colorName": "あざとブラウン",
          "isCorrect": false
        }
      ],
      "hint1": {
```

```

        "dia": "14.5",
        "gdia": "13.6",
        "bc": "8.6"
    },
    "hint2": {
        "comment": "フチ×軸固ブラウンで工いらすのぶるんとした瞳に。"
    }
}
// ... 残り9問
]
}

```

【POST】 回答送信: `action=submit`

リクエスト例:

```

{
  "action": "submit",
  "user": "USER001",
  "store": "渋谷店",
  "score": 85,
  "correct": 8,
  "total": 10,
  "accuracy": 0.8,
  "timeSec": 180,
  "hints": 3,
  "dateISO": "2025-10-06T10:30:00Z",
  "mode": "daily",
  "details": [
    {
      "questionNo": 1,
      "correctAnswer": "ABC123|ダスティティ―|クレイジーブラウン",
      "userAnswer": "ABC123|ダスティティ―|クレイジーブラウン",
      "isCorrect": true,
      "timeSec": 15,
      "hintsUsed": 0
    }
  ]
  // ... 残り9問
]
}

```

レスポンス:

```

{
  "success": true,
  "message": "回答を保存しました",
  "historyId": "HIST20251006_USER001_001"
}

```

【GET】 ランキング: `?action=ranking&scope=daily`

レスポンス例:

```
{
  "scope": "daily",
  "date": "2025-10-06",
  "rankings": [
    {
      "rank": 1,
      "userId": "USER003",
      "name": "みゆちゃん",
      "store": "新宿店",
      "points": 980,
      "accuracy": 0.98,
      "attempts": 1
    }
    // ...
  ]
}
```

【GET】マイ成績: `?action=mystats&user=USER001`

レスポンス例:

```
{
  "userId": "USER001",
  "name": "さくらちゃん",
  "store": "渋谷店",
  "summary": {
    "totalPlayed": 15,
    "totalPoints": 1275,
    "avgPoints": 85,
    "avgAccuracy": 0.85,
    "avgTime": 170,
    "totalHints": 45
  },
  "recent10": [
    {
      "date": "2025-10-06",
      "mode": "daily",
      "score": 85,
      "accuracy": 0.8,
      "timeSec": 180,
      "hints": 3
    }
    // ... 残り9回
  ]
}
```

## 7. UI/UXガイドライン

### 7.1 デザイン仕様

カラーパレット:

- **メイン:** #FF6EC7（ホテラバピンク）
- **サブ:** #FF9472
- **背景:** グラデーション（#fff2f7 → #fffafe）

#### フォント:

- **必須:** Noto Sans JP（woff2同梱）
- **用途:** 全画面・グラフ・図

#### レイアウト:

- **前提:** スマホファースト
- **ナビ:** 下部ナビ（ホーム／学習／ランキング／マイページ／設定）
- **余白:** 明瞭な段差と余白

## 7.2 画面構成

| 画面    | 主要要素                          |
|-------|-------------------------------|
| ログイン  | ロゴ・新規登録ボタン・ログインボタン            |
| ホーム   | ユーザー情報・クイズ開始ボタン・ランキングへのリンク    |
| クイズ   | 問題番号・タイマー・レンズ画像×2・4択・ヒントボタン×2 |
| 結果    | スコア・正答数・平均時間・間違えた問題・ホームに戻る    |
| ランキング | 日/週/月タブ・順位リスト・自分の順位ハイライト      |
| マイ成績  | 直近10回グラフ（Canvas）・通算サマリ        |

## 8. 実装詳細（コード例）

### 8.1 GAS関数サンプル

#### Config.gs（設定定義）

```
// filename: Config.gs
const CONFIG = {
  SHEET_IDS: {
    MASTER: '1Uf2e0eXwcsQGjFtTtEeAWuYh74lh4fFE4NdjmyKHrj0',
    USERS: '1X0TyeL1zER6xIceUDSbJX-GFbqvi2orAiSWHRXIC7M',
    HISTORY: '1ShWXLvY9RimRYfsAkwoRyM2Bfwj4a3zVmr5bQc33-o0',
    RANKINGS: '1l2REcy2v5OpyzoY3k61kCzJ3SYKOBBCMxTLCeHWutT8'
  },
  COLS: {
    MASTER: {
      E: 5, // 元品番（1始まり）
      I: 9, // ブランド（カナ）
      J: 10, // カラー（カナ）
      K: 11, // 装用期間
      P: 16, // DIA
      Q: 17, // G.DIA
      R: 18, // BC
      AK: 37, // コメント
      AJ: 36 // カラーカテゴリ
    }
  }
},
```

```

GITHUB: {
  USER: 'y4m4usr',
  REPO: 'HL001-quiz-karacon-academia-new',
  BRANCH: 'main',
  LENS_PATH: 'imagesnew1/lens/lens1/',
  SAMUNE_PATH: 'imagesnew1/samune/samune1/'
}
};

```

## DataAccess.gs (データ読み込み)

```

// filename: DataAccess.gs

// 空白判定
const isBlank = v => v === null || v === undefined || String(v).trim() === "" || v === "-";

// マスター読み込み (3行目以降)
function readMaster_() {
  const sh = SpreadsheetApp.openById(CONFIG.SHEET_IDS.MASTER).getSheetByName('master');
  const values = sh.getRange(3, 1, sh.getLastRow()-2, sh.getLastColumn()).getValues();
  const COL = CONFIG.COLS.MASTER;

  const rows = [];
  for (const v of values) {
    // 空白セルが1つでもあれば除外
    if (v.some(isBlank)) continue;

    const r = {
      E: v[COL.E-1],
      I: v[COL.I-1],
      J: v[COL.J-1],
      K: v[COL.K-1],
      P: v[COL.P-1],
      Q: v[COL.Q-1],
      R: v[COL.R-1],
      AK: v[COL.AK-1],
      AJ: v[COL.AJ-1]
    };

    // 必須列チェック
    if ([r.E, r.I, r.J, r.K, r.AJ].some(isBlank)) continue;

    rows.push(r);
  }

  return rows;
}

// 画像URL生成
function buildImageUrl_(E, I, J, K, type) {
  // type: 'lens' or 'samune'
  const filename = sanitizeFilename(`${E}_${I}_${J}_${K}_${type}.jpg`);
}

```

```
const path = type === 'lens' ? CONFIG.GITHUB.LENS_PATH : CONFIG.GITHUB.SAMUNE_PATH;

return `https://raw.githubusercontent.com/${CONFIG.GITHUB.USER}/${CONFIG.GITHUB.REPO}/${CONFIG.GITHUB.BRANCH}/${path}${filename}`;
}

// ファイル名サニタイズ
function sanitizeFilename_(str) {
  return str
    .replace(/s+/g, '_')
    .replace(/[^w\u3040-\u309F\u30A0-\u30FF\u4E00-\u9FAF.-]/g, '_')
    .replace(/_+/g, '_');
}
```

QuizLogic.gs (問題生成・採点)

```
// filename: QuizLogic.gs

// 色語分割
const splitColors = s =>
  String(s || "")
    .split(/[,\u3001\uFF0C\u30FB\/\|]+/)
    .map(x => x.trim())
    .filter(x => x);

// E|I|J キー生成
const ck = r => [r.E, r.I, r.J].join("|");

// 問題生成 (10問)
function generateQuestions_(count = 10) {
  const allItems = readMaster_();
  if (allItems.length < 4) {
    throw new Error('データ不足: 最低4件必要');
  }

  const questions = [];
  const usedKeys = new Set();

  // 10問選定 (E|I|J重複除外)
  while (questions.length < count && allItems.length > 0) {
    const randomIndex = Math.floor(Math.random() * allItems.length);
    const candidate = allItems[randomIndex];
    const key = ck(candidate);

    if (!usedKeys.has(key)) {
      questions.push(candidate);
      usedKeys.add(key);
    }
  }

  allItems.splice(randomIndex, 1);
}
```

```

// 各問題に誤答3件を生成
const allData = readMaster_();
return questions.map((correct, idx) => {
  const wrongs = selectWrongAnswers_(correct, allData);
  return makeQuestion_(correct, wrongs, idx + 1);
});
}

// filename: QuizLogic.gs

// 誤答3件抽出（必ず3件返却）
function selectWrongAnswers_(correct, pool, n = 3) {
  const correctKey = ck(correct);
  const correctCategories = splitColors(correct.AJ);

  // 第1優先: カテゴリ一致
  let cans = pool.filter(r => {
    if (ck(r) === correctKey) return false;
    const itemCategories = splitColors(r.AJ);
    return itemCategories.some(cat => correctCategories.includes(cat));
  });

  cans = cans.sort(() => Math.random() - 0.5);

  const picked = [];
  for (const r of cans) {
    if (picked.length < n) picked.push(r);
  }

  // 第2優先: フォールバック（カテゴリ無視）
  if (picked.length < n) {
    const remaining = pool.filter(r =>
      ck(r) !== correctKey && !picked.includes(r)
    );
    remaining.sort(() => Math.random() - 0.5).forEach(r => {
      if (picked.length < n) picked.push(r);
    });
  }

  // ★ 第3優先: 完全ランダム抽出（新規追加）
  if (picked.length < n) {
    // E||J の重複チェックのみ
    const usedKeys = new Set([correctKey, ...picked.map(p => ck(p))]);

    // まずは E||J が異なるものから抽出
    let additionalCands = pool.filter(r => !usedKeys.has(ck(r)));
    additionalCands.sort(() => Math.random() - 0.5);

    for (const r of additionalCands) {
      if (picked.length < n) {
        picked.push(r);
        usedKeys.add(ck(r));
      }
    }
  }
}

```

```

    }

    // それでも足りない場合、装用期間K違いも許容 (E|I|J|K で異なる)
    if (picked.length < n) {
      const allAvailable = pool.filter(r => {
        const fullKey = `${ck(r)}|${r.K}`;
        const correctFullKey = `${correctKey}|${correct.K}`;
        const pickedFullKeys = picked.map(p => `${ck(p)}|${p.K}`);

        return fullKey !== correctFullKey && !pickedFullKeys.includes(fullKey);
      });

      allAvailable.sort(() => Math.random() - 0.5);

      for (const r of allAvailable) {
        if (picked.length < n) picked.push(r);
      }
    }

    return picked.slice(0, n);
  }

  // 1問の構築
  function makeQuestion_(correct, wrongs, qNum) {
    const options = [
      {
        id: 1,
        brandName: correct.I,
        colorName: correct.J,
        isCorrect: true
      },
      ...wrongs.map((w, i) => ({
        id: i + 2,
        brandName: w.I,
        colorName: w.J,
        isCorrect: false
      })))
    ].sort(() => Math.random() - 0.5);

    return {
      questionNumber: qNum,
      lensImageUrl: buildImageUrl_(correct.E, correct.I, correct.J, correct.K, 'lens'),
      thumbnailImageUrl: buildImageUrl_(correct.E, correct.I, correct.J, correct.K, 'samune'),
      correctAnswer: {
        E: correct.E,
        I: correct.I,
        J: correct.J,
        K: correct.K
      },
      options: options,
      hint1: {
        dia: correct.P || "",

```

```

    gdia: correct.Q || "",
    bc: correct.R || ""
  },
  hint2: {
    comment: correct.AK || ""
  }
};
}

// スコア計算
function calculateScore_(correct, total, hints) {
  const incorrect = total - correct;
  let score = 100 - (incorrect * 10) - (hints * 3);
  return Math.max(0, Math.min(100, score));
}

```

## 8.2 フロント連携

### API呼び出し例 (JavaScript)

```

// filename: main.js

const API_URL = 'https://script.google.com/macros/s/{SCRIPT_ID}/exec';

// 問題取得
async function fetchQuestions() {
  try {
    const response = await fetch(`${API_URL}?action=items`);
    const data = await response.json();
    return data.questions;
  } catch (error) {
    console.error('問題取得エラー:', error);
    return [];
  }
}

// 回答送信
async function submitAnswers(data) {
  try {
    const response = await fetch(API_URL, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        action: 'submit',
        ...data
      })
    });
    return await response.json();
  } catch (error) {
    console.error('送信エラー:', error);
  }
}

```

```

    return { success: false };
  }
}

// 画像読み込み（フォールバック付き）
async function loadImage(url, placeholderUrl) {
  return new Promise((resolve, reject) => {
    const img = new Image();
    img.onload = () => resolve(url);
    img.onerror = () => {
      console.warn('画像取得失敗、プレースホルダー使用:', url);
      resolve(placeholderUrl);
    };
    img.src = url;
  });
}

```

## 修正内容のまとめ

項目変更前変更後最低保証データ不足時、3件未満の可能性あり必ず3件の誤答を返却フォールバック段階2段階（カテゴリー一致 → カテゴリー無視）3段階（カテゴリー一致 → カテゴリー無視 → 完全ランダム）同一商品の扱いE||J一致は厳格に除外最終手段としてK違いも許容エラーハンドリングデータ不足でエラーの可能性必ず出題可能

## テストケース

極端なデータ不足の例:

EIJKAJABC123ブランドAカラーX1dayブラウンABC123ブランドAカラーX1monthブラウンDEF456ブランドBカラーY1dayピンク

正答: ABC123 | ブランドA | カラーX | 1day

誤答抽出結果:

1. ☒ DEF456 | ブランドB | カラーY | 1day（E||J異なる）
2. ☒ ABC123 | ブランドA | カラーX | **1month**（K違い・同一商品）
3. ☒ DEF456 | ブランドB | カラーY | 1day（重複・最終手段）

→ データが2件しかなくても必ず3件の選択肢を生成

再試行

## 8.3 エラーハンドリング

よくあるエラーと対処

| エラー            | 原因           | 対処                      |
|----------------|--------------|-------------------------|
| 画像が表示されない      | GitHub画像が404 | ファイル名の確認・再アップロード        |
| スプレッドシート読み込み失敗 | 権限不足         | GASプロジェクトの実行ユーザー確認      |
| HISTORYに書き込めない | 編集権限なし       | シート共有設定で編集権限付与          |
| 10問生成できない      | データ不足        | master タブでE/I/J/K/AJを補完 |

## 8.4 実装チェックリスト

### データ取得

- ☐ スプレッドシート3行目以降を取得
- ☐ E, I, J, AJ が でない行のみ
- ☐ 列インデックスで参照（1始まり → 0始まり変換）

### 画像取得

- ☐ GitHub raw URL で取得
- ☐ ファイル名から E||J|K 抽出
- ☐ レンズとサムネのペア確保

### 問題生成

- ☐ 10問ランダム選定
- ☐ E||J 重複チェック
- ☐ 誤答3件生成（カテゴリー一致優先）
- ☐ フォールバック処理

### API実装

- ☐ GET /items
- ☐ POST /submit（履歴保存）
- ☐ GET /ranking
- ☐ GET /mystats

### UI実装

- ☐ Noto Sans JP 適用
- ☐ レンズ画像2枚表示
- ☐ 4択ボタン
- ☐ ヒントボタン×2
- ☐ 正解時サムネ表示（2秒→自動遷移）
- ☐ Canvas グラフ（マイ成績）

## 付録: 参考資料

### A. 旧仕様書との対応

| 旧バージョン | 主な変更内容            | v2.1.1での反映 |
|--------|-------------------|------------|
| v1.7   | API仕様・スコアリング・画像運用 | 統合済み       |
| v2.0   | 問題生成ロジック・E  J重複除外 | 統合済み       |

### B. 画像管理フロー図

#### 【手動運用フロー】

- 109販促CPのmasterタブに商品情報を追加
- GitHubに画像を手動アップロード（ファイル名: E\_L\_J\_K\_lens.jpg）
- GASが次回の問題生成時に自動認識

## C. トラブルシューティング

### 問題が生成できない場合:

1. `master` タブでE/I/J/K/AJ列を確認
2. が含まれていないか確認
3. GitHub画像が存在するか確認

### 画像が表示されない場合:

1. ファイル名が正しいか確認（スペース・記号の扱い）
2. GitHub上に実際にファイルが存在するか確認
3. ブラウザのNetwork タブでURLを確認

---

### 以上が v2.1.1 最終仕様書です。

運用者が全体構造を理解しやすいよう、ファイル一覧・構造図を先頭に配置し、実装詳細（コード例）は最後にまとめました。全リソースの編集権限が統一されているため、スムーズな開発・運用が可能です。